

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Stripe (Payment Processing)

Delivery No. 2 — Software Architecture

Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Course: Cloud Computing

Professor: Carlos Andres Sierra Virguez

November 2, 2026

Cristian Santiago Lopez Cadena	20222020027
Xiomara Salomé Arias Arias	20222020028
Juan Diego Grajales Castillo	20221020128
Arlo Nicolás Ocampo Gallego	20221020104

Contents

1 Executive Summary **3**

2 System Context Diagram **4**

3 Architecture Diagram **6**
3.1 Purpose . . . 6
3.2 Two independent groupings . . . 7
3.3 Services and data ownership . . . 7
3.4 Interaction topology . . . 8
3.5 Critical path of payment authorization (Use Case 01) . . . 9

4 Service Catalog **10**
4.1 Purpose and Design Principles . . . 10
4.2 Service Overview . . . 11
4.3 Edge Gateway . . . 12
4.4 Authentication Service . . . 12
4.5 Tokenization Service . . . 13
4.6 Network Gateway . . . 14
4.7 Payment Orchestrator . . . 14
4.8 Fraud Scoring Service . . . 15
4.9 Ledger Service . . . 16
4.10 Refund and Dispute Service . . . 17
4.11 Settlement and Reconciliation Service . . . 18
4.12 Monitoring and Audit Service . . . 18
4.13 Notification and Webhook Dispatcher . . . 19
4.14 Read Models and Query Service . . . 20
4.15 Cross-Service Data Ownership and Consistency . . . 21
4.16 Scaling and Capacity Considerations . . . 21

5 Data Architecture **22**
5.1 Data Ownership Model . . . 22
5.2 Storage Strategy per Service . . . 23
5.3 Data Model . . . 25
5.4 Consistency Requirements . . . 27
5.5 Transaction Boundaries . . . 28
5.6 Shared-Data Anti-Patterns and How They Are Avoided . . . 29

6 Communication Architecture **30**
6.1 Selection Criteria and Protocols . . . 30
6.2 Communication Diagram . . . 30
6.3 Communication Matrix . . . 31
6.4 Synchronous Interactions . . . 33
6.5 Asynchronous Interactions . . . 36
6.6 Justification of the Asynchronous Choices . . . 37

7 Architecture Decision Records **39**
7.1 Architecture Decision Record 1: Coarse-grained services aligned with the Delivery 1 modules . . . 39
7.2 Architecture Decision Record 2: Event-driven propagation after the payment commit, using a transactional outbox . . . 40
7.3 Architecture Decision Record 3: Synchronous authorization with an explicit pending state . . . 41
7.4 Architecture Decision Record 4: Distributed relational storage for payments and the ledger; key-value and columnar stores elsewhere . . . 41
7.5 Architecture Decision Record 5: Command Query Responsibility Segregation with event-fed read models . . . 42
7.6 Architecture Decision Record 6: Orchestrated sagas for refunds and disputes instead of distributed transactions . . . 43

8 Requirements Traceability **44**

1 Executive Summary

Architecture Approach

The system is designed as a distributed, modular service-oriented architecture, directly aligned with the functional modules established in Delivery 1 (Authentication, Payment Processing, Fraud Detection, Settlement and Reconciliation, Refund and Dispute Management, and Monitoring and Audit). To meet the strict latency and availability requirements under peak load, the architecture avoids distributed ACID transactions (two-phase commits) across services. Instead, it relies on localized ACID transactions within single-owner data stores, orchestrated sagas with compensating actions for long-running workflows (e.g., refunds and disputes), and a transactional outbox pattern to guarantee reliable asynchronous event propagation.

Key Design Principles

To ensure that each capability can be developed, deployed, scaled, and recovered independently, the architecture enforces eight core design principles across all services:

- **Single ownership of data:** Each service is the exclusive writer to its own data store. Other services access this data strictly through synchronous API calls, asynchronous event subscriptions, or dedicated read models, never by querying another service's database directly.
- **Idempotent processing:** Operations that can be retried—particularly payment creation, refunds, and event consumption—are strictly idempotent. Clients must provide idempotency keys, and consumers deduplicate by event identifiers to ensure financial effects are never duplicated.
- **Financial integrity:** Money movements are posted as balanced, double-entry journals. They are committed atomically inside the authoritative ledger and are strictly immutable; any corrections require explicit reversing journals rather than in-place updates.
- **Asynchronous integration after commit:** The synchronous critical path is restricted solely to the steps necessary to authorize a payment. All downstream processes (ledger posting, external notifications, audit trailing, and read-model updates) run asynchronously via a shared Event Log to decouple consumers and absorb traffic peaks.
- **Independent scalability:** Stateless components (e.g., gateways, scoring engines) scale horizontally behind load balancers based on request rates and CPU utilization. Stateful stores are scaled using strategies that preserve their required consistency and durability guarantees (e.g., sharding by seller ID).
- **Failure isolation:** A slow or unavailable downstream dependency must not trigger a cascading failure in the payment path. Every synchronous call enforces strict bounded deadlines, circuit breakers, and predefined fallback mechanisms.
- **Minimal cardholder-data scope:** To satisfy strict PCI DSS compliance, the Primary Account Number (PAN) is strictly confined to the Cardholder Data Environment (CDE). No other internal service ever stores or processes the PAN; they operate exclusively on secure tokens.

- **Security by design:** The platform authenticates every request, enforces Role-Based Access Control (RBAC), and encrypts data both at rest and in transit (using TLS 1.3 externally and mutual TLS for inter-service communication).

2 System Context Diagram

The following diagram shows the Stripe platform as a single black box, together with all external actors and systems that interact with it. It is consistent with the actor and use case catalog defined in Delivery 1 (US-01 to US-20). Every arrow is labeled with the nature of the communication it represents.

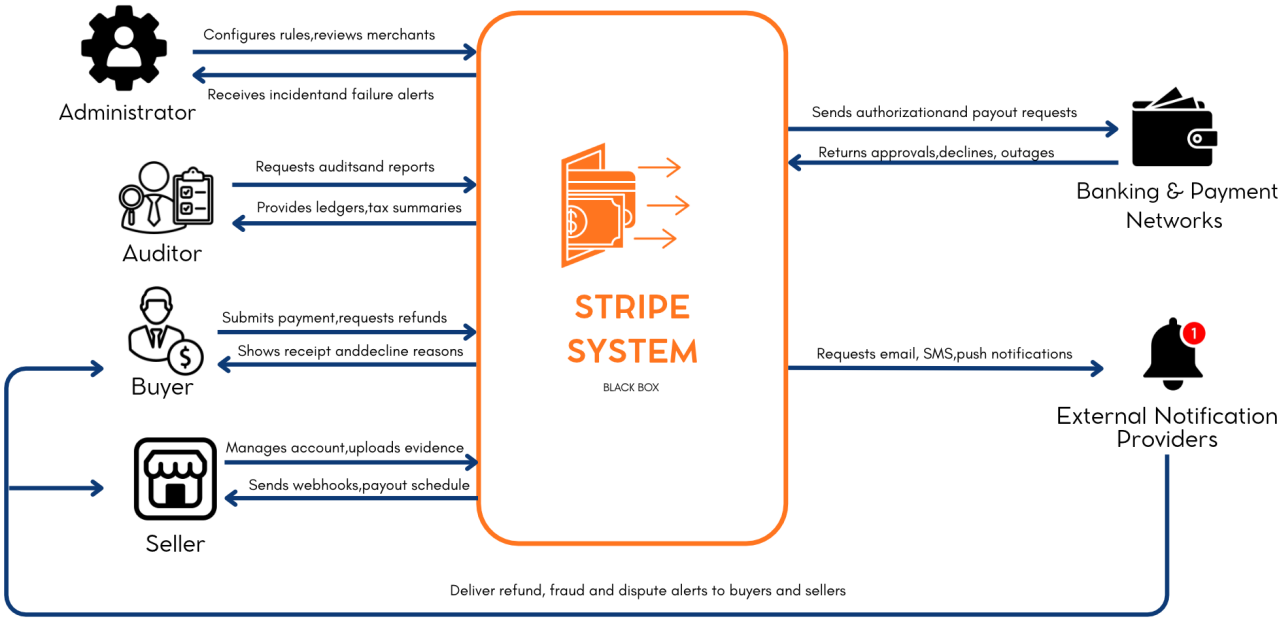


Figure 1: System Context Diagram (C4 Level 1) of the Stripe platform.

External Elements

Table 1 describes each external element shown in Figure 1: what it is, how it interacts with the system, and how often the interaction occurs.

Table 1: External elements of the Stripe system and their interactions.

Element	What it is	How it interacts with the system	Frequency	User stories
Buyer	End customer who pays for purchases in online stores.	*Sends:* submits payment details and requests refunds. *Receives:* purchase receipt on screen and plain-language reasons when a payment is declined.	Continuous, once per purchase attempt (thousands per minute at global scale).	US-01, US-02, US-05
Seller (Merchant)	Business that sells products or services through the platform.	*Sends:* manages its account, views balance and payout schedule, and uploads dispute evidence. *Receives:* payment-success webhooks, payout schedules, and dispute deadline warnings.	Webhooks in real time ($\leq$5 s per payment); dashboard queries several times per day; payouts on a periodic schedule.	US-06 to US-11
Administrator	Internal operations, support, and risk staff of the platform.	*Sends:* configures security rules (bot blocking, failover) and reviews merchant verification cases. *Receives:* incident alerts, regional failure-rate alerts, and bank outage notifications.	Continuous monitoring; review and alert handling on an event basis.	US-12 to US-16
Auditor	Accounting and compliance staff responsible for financial and regulatory review.	*Sends:* requests audits, reconciliation reports, and tax processing jobs. *Receives:* immutable ledgers, annual tax summaries, PCI-DSS evidence, and flagged money-laundering cases.	Monthly (reconciliation), annually (tax summaries), and on an event basis (compliance alerts).	US-17 to US-20
Banking & payment networks	External banks, card networks, and acquirers that authorize and settle payments.	*Sends (from system):* authorization, capture, and payout requests. *Receives (to system):* approvals, declines, and timeout/error signals used to detect outages and trigger failover.	Continuous, once per transaction (response within seconds); error rates monitored over rolling 2-minute windows.	US-01, US-05, US-09, US-15, US-16
External notification providers	Third-party email, SMS, and push notification services.	*Sends (from system):* requests delivery of alerts and confirmations. *Delivers:* refund emails, fraud alerts, subscription retry reminders, and dispute warnings to Buyers and Sellers.	Event-driven: fraud alerts $\leq$30 s, refund emails $\leq$5 min, dispute warnings 48 h before the deadline.	US-03, US-04, US-10, US-11

Interaction Summary

All interactions occur through the system boundary, since the platform is treated as a black box at this level:

- **Human actors** (Buyer, Seller, Administrator, Auditor) interact through the platform's web interfaces, dashboards, and APIs.
- **Banking & payment networks** exchange authorization and settlement messages with the system in both directions.
- **External notification providers** receive delivery requests from the system and forward alerts to Buyers and Sellers; the platform does not send email, SMS, or push messages directly.

3 Architecture Diagram

3.1 Purpose

The diagram opens the black box shown in the system context diagram of Delivery 1 and answers three questions: which services exist, how they are grouped, and how they interact. It is a container-level view: each rounded box is a separately deployable service, and each cylinder is a data store owned by exactly one service.

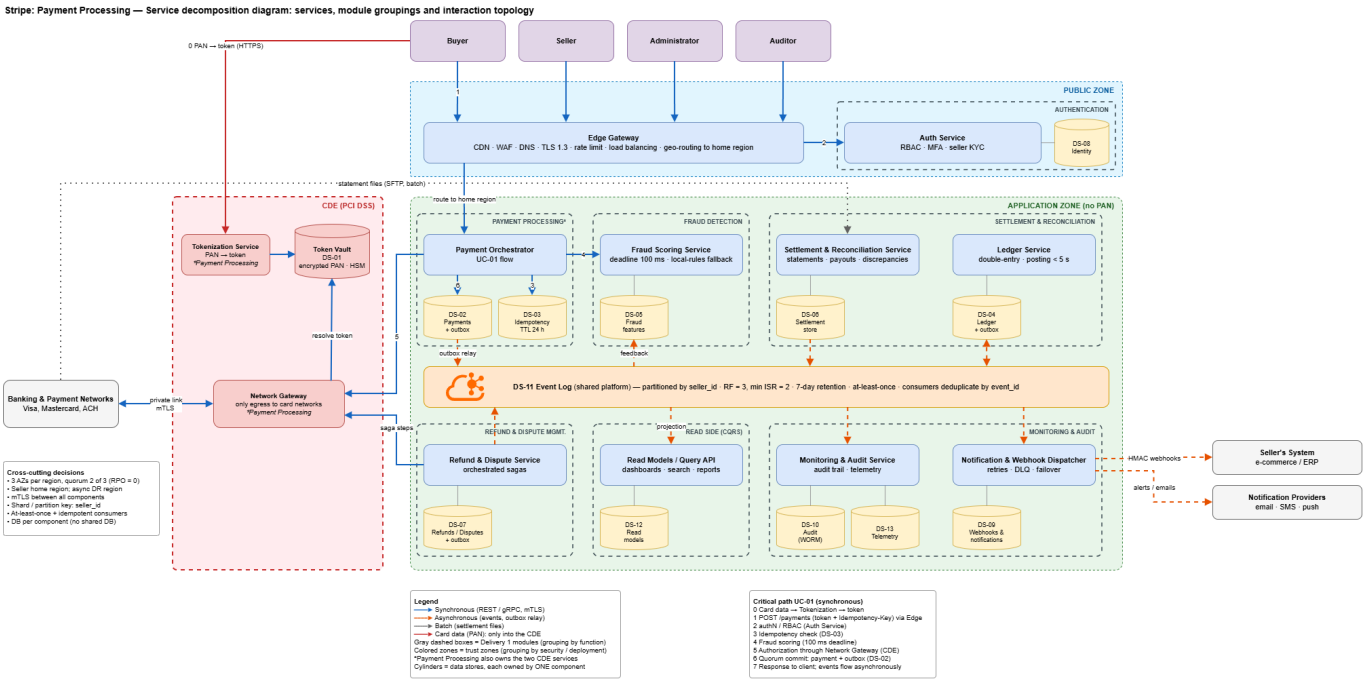


Figure 2: Architecture Diagram: Services, module groupings and interaction topology

3.2 Two independent groupings

The diagram shows two groupings at the same time, because they answer different questions.

- **By module (gray dashed boxes)**. The six modules defined in Delivery 1: Authentication, Payment Processing, Fraud Detection, Settlement and Reconciliation, Refund and Dispute Management, and Monitoring and Audit, plus a shared read side. This grouping answers: who is responsible for which business capability?
- **By trust zone (colored backgrounds).** The public zone (light blue), the application zone (green) and the Cardholder Data Environment (red). This grouping answers: where does each service run, and how strictly is it protected?

3.3 Services and data ownership

Table 2: Services and data ownership

Service	**Module**	**Zone**	**Responsibility**	**Data store owned**
Edge Gateway	Managed edge (part of the system boundary)	Public	Content delivery network, web application firewall, domain name resolution, Transport Layer Security termination, rate limiting, routing to the seller's home region	None (stateless)
Authentication Service	Authentication	Public	Role-based access control, multi-factor authentication, seller verification (Know Your Customer data)	Identity Store
Tokenization Service	Payment Processing	Cardholder Data Environment	Exchanges the Primary Account Number for a token	Token Vault
Network Gateway	Payment Processing	Cardholder Data Environment	Only exit toward the banking and payment networks	None (resolves tokens in the Token Vault)
Payment Orchestrator	Payment Processing	Application	Coordinates payment authorization (Use Case 01)	Payment Store with outbox; Idempotency Store
Fraud Scoring Service	Fraud Detection	Application	Risk decision within 100 milliseconds; local-rules fallback	Fraud Feature Store

Continue in the next page

Table 2 – *continuation of the previous page*

Service	**Module**	**Zone**	**Responsibility**	**Data store owned**
Settlement and Reconciliation Service	Settlement and Reconciliation	Application	Network statements, payouts, discrepancies	Settlement Store
Ledger Service	Settlement and Reconciliation	Application	Posts double-entry journals from events	Ledger with outbox
Refund and Dispute Service	Refund and Dispute Management	Application	Orchestrated sagas for refunds and disputes	Refund and Dispute Store with outbox
Monitoring and Audit Service	Monitoring and Audit	Application	Immutable audit trail and telemetry	Audit Store (write-once); Telemetry Store
Notification and Webhook Dispatcher	Monitoring and Audit	Application	Signed webhooks to seller systems; notifications through external providers	Notification and Webhook Store
Read Models and Query Service	Shared read side	Application	Dashboards, transaction search and reports built from events	Read Model Store

The Event Log is a shared platform component: a partitioned, replicated log that carries every domain event between services. Data store labels printed inside the diagram file: 01 Token Vault, 02 Payment Store, 03 Idempotency Store, 04 Ledger, 05 Fraud Feature Store, 06 Settlement Store, 07 Refund and Dispute Store, 08 Identity Store, 09 Notification and Webhook Store, 10 Audit Store, 11 Event Log, 12 Read Model Store, 13 Telemetry Store. In the file they appear as “DS” (Data Store) followed by the number.

3.4 Interaction topology

Table 3: Interaction topology

Interaction style	**Drawn as**	**Used for**	**Why**
Synchronous request	Solid blue arrow	Representational State Transfer interfaces from clients; binary remote procedure calls over Hypertext Transfer Protocol version 2 with mutual Transport Layer Security between services	The Buyer waits for the answer, so only the steps that decide the outcome are synchronous
Asynchronous event	Dashed orange arrow	Domain events published through the transactional outbox to the Event Log	Decouples consumers, absorbs traffic peaks, allows replay
Batch file	Dotted gray arrow	Settlement statements from the banking networks, delivered by Secure File Transfer Protocol	The networks deliver settlement as periodic files
Card data	Solid red arrow	Primary Account Number sent only to the Tokenization Service	No other component may ever see the card number
Data ownership	Thin gray line	Service to its own data store	Each store has exactly one owner; no shared databases

3.5 Critical path of payment authorization (Use Case 01)

1. The Buyer sends card data directly to the Tokenization Service and receives a token.
2. The Buyer submits the payment request, with the token and an idempotency key, through the Edge Gateway.
3. The Edge Gateway validates the access token with the Authentication Service.
4. The Payment Orchestrator checks the idempotency key; a repeated key returns the stored response and nothing runs twice.
5. The Payment Orchestrator asks the Fraud Scoring Service for a risk decision with a 100-millisecond deadline.
6. The Payment Orchestrator requests authorization through the Network Gateway, which resolves the token in the Token Vault and contacts the banking network.
7. The Payment Orchestrator commits the payment state, the status transition and the outbox record in one transaction, replicated to a majority of availability zones.

8. The response returns to the Buyer. From here on, everything (ledger posting, notifications, audit, read models) happens asynchronously through the Event Log.

The estimated budget at the 95th percentile is about 220 milliseconds, below the 500-millisecond limit of Non-Functional Requirement 05.

4 Service Catalog

This section defines, for each service of the architecture in Section 3, its responsibility, the public interfaces it exposes, the data it owns, the services it depends on, and how it scales. The catalog uses exactly the service names, data stores (DS-01 to DS-13), trust zones, and interaction identifiers (S-xx, A-xx, B-xx) introduced in the architecture, data, and communication sections, so the four views describe one single system. It is cloud-independent: it names patterns and protocols, not products. These boundaries describe the architectural design proposed for this project and do not imply that the production Stripe platform uses the same internal decomposition.

4.1 Purpose and Design Principles

The service catalog establishes clear service boundaries so that each capability can be developed, deployed, scaled, and recovered independently. The decomposition follows the six Delivery 1 modules plus a shared read side, as decided in Architecture Decision Record 1. Eight principles govern every service in the catalog:

- **Single ownership of data.** Each service is the only writer of its data store; other services reach that data through an API call, an event subscription, or a read model, never by touching another store directly (Section 5.1).
- **Idempotent processing.** Operations that can be retried, in particular payment creation, refunds, and event consumers, must not produce duplicate financial effects. Clients provide an idempotency key; consumers deduplicate by `event_id` (NFR-07, FR-02).
- **Financial integrity.** Money movements are posted as balanced double-entry journals, committed atomically inside the authoritative store, and never updated in place (FR-07, ADR-4).
- **Asynchronous integration after commit.** Only the steps that decide a payment's outcome are synchronous; ledger posting, notifications, audit, read models, and fraud feedback run asynchronously through the Event Log (ADR-2, ADR-3, NFR-05).
- **Independent scalability.** Stateless services scale horizontally behind a load balancer; stateful stores scale with strategies that preserve their consistency and durability guarantees (ADR-4).
- **Failure isolation.** A slow or unavailable downstream dependency must not cascade into the payment path; every synchronous call carries a deadline, a circuit breaker, and a defined fallback (ADR-3, NFR-06).
- **Minimal cardholder-data scope.** The Primary Account Number (PAN) lives only inside the Cardholder Data Environment (CDE); no other service ever sees it (NFR-11, ADR-1).

- **Security by design.** Services authenticate every request, enforce role-based authorization, encrypt data in transit (TLS 1.3, mutual TLS between services) and at rest, and never expose sensitive payment data unnecessarily (NFR-11).

4.2 Service Overview

Table 4 summarizes the eleven services and the shared Event Log. The Edge Gateway and the Network Gateway are stateless and own no store; the Event Log (DS-11) is shared *infrastructure*, not a service, and never acts as a system of record.

Table 4: Service overview: module, trust zone, responsibility, and scaling driver.

Service	Zone	Primary responsibility	Scaling driver
Edge Gateway	Public	Admission, TLS termination, WAF, routing, rate limiting	Request rate (stateless)
Authentication Service	Public	Identity, RBAC, MFA, seller verification (KYC)	Request rate
Tokenization Service	CDE	Exchange PAN for a token	Tokenization rate
Network Gateway	CDE	Sole exit toward the banking and payment networks	Authorization rate
Payment Orchestrator	Application	Payment lifecycle, idempotency, authorization orchestration	Request volume, DB writes
Fraud Scoring Service	Application	Risk decision within 100 ms; local-rules fallback	Inference latency, request rate
Ledger Service	Application	Authoritative double-entry financial records	Event volume per seller shard
Settlement and Reconciliation Service	Application	Payouts, statement matching, discrepancies	Batch/file volume
Refund and Dispute Service	Application	Orchestrated sagas for refunds and disputes	Saga volume, in-flight disputes
Monitoring and Audit Service	Application	Immutable audit trail and telemetry	Event/telemetry ingestion rate
Notification and Webhook Dispatcher	Application	Signed webhooks and external notifications	Queue depth, endpoint latency
Read Models and Query Service	Application	Dashboards, transaction search, reports	Query rate
Event Log (DS-11)	Application	Shared durable pub/sub (infrastructure, not a service)	Partition count, consumer lag

Each service below is described with five attributes: responsibility (what it does and does not do), representative APIs, data owned, dependencies, and scaling characteristics.

4.3 Edge Gateway

Responsibility. The Edge Gateway is the single public entry point. It terminates TLS 1.3, applies the web application firewall, resolves the domain name, enforces rate limiting, validates the basic structure of each request, propagates a correlation identifier, and routes the request to the seller's home region and to the right backend service. It does *not* implement financial business logic or decide payment outcomes.

Representative APIs. The gateway exposes the public REST surface over HTTPS and forwards internally over gRPC with mutual TLS. It is the public face of the payment endpoints handled by the Payment Orchestrator (S-02):

Method	Path	Purpose
POST	`/v1/payments`	Admit and route a payment request (requires `Idempotency-Key`).
GET	`/v1/payments/{id}`	Route a status query.
POST	`/v1/refunds`, `/v1/disputes/{id}/responses`	Route refund and dispute operations.
GET	`/v1/payments`, `/v1/balance`, `/v1/settlements`	Route read-side queries.
GET	`/healthz`	Internal health and readiness (not publicly exposed).

Data owned. None. The gateway is stateless; routing rules and rate-limit policies are configuration, and any shared counter used for distributed rate limiting is external infrastructure, not a system of record.

Dependencies. Authentication Service (S-03, token and RBAC validation before forwarding), Payment Orchestrator (S-02), Read Models and Query Service (S-08), and the other application services it fronts.

Scaling characteristics. Stateless; scales horizontally behind a load balancer driven by request rate, concurrent connections, CPU, and latency percentiles. It must shed load predictably (reject excess traffic) rather than exhaust downstream capacity, and distributed rate limiting must not turn a shared counter into a bottleneck. It fails closed: if authentication cannot be verified, the request is rejected (S-03).

4.4 Authentication Service

Responsibility. Owns identity and access: role-based access control (RBAC), multi-factor authentication, and seller verification (Know Your Customer data) for FR-11 and FR-12 (UC-09). It answers "who is the caller and what may they do" and records verification decisions. It does *not* store payment or financial data.

Representative APIs.

Method / RPC	Operation	Purpose
gRPC	`ValidateToken`	Resolve the identity and roles of a caller (S-03).
POST	`/v1/sellers/{id}/verification`	Submit seller verification information (FR-11).
POST	`/v1/verifications/{id}/decision`	Approve, reject, or request more information (FR-12).

Data owned. Identity Store (DS-08): users, roles, MFA factors, sellers, and their verification data and documents. It publishes `seller.verification_updated` (A-09); other services keep only the opaque `seller_id`.

Dependencies. Consumed synchronously by the Edge Gateway (S-03) and by Seller and Administrator clients (S-16). Emits events to the Notification and Webhook Dispatcher, Monitoring and Audit, and Read Models (A-09).

Scaling characteristics. Stateless application tier scaling on request rate; `ValidateToken` is read-heavy and cacheable with a short time to live, giving a sub-10 ms P95 (S-03). It uses strong consistency so a revoked role or a rejected seller takes effect at once (Section 5.4).

4.5 Tokenization Service

Responsibility. Inside the CDE, it exchanges the Primary Account Number for a token and resolves a token back to the PAN only for the Network Gateway. It is the *only* component that ever sees card numbers, which keeps the PCI DSS scope minimal (NFR-11, ADR-1). It does *not* authorize payments or keep business state.

Representative APIs.

Method / RPC	Operation	Purpose
POST	`/v1/tokens` (`Tokenize`)	Exchange card data for a token; called directly by the Buyer client so the PAN never transits the application zone (S-01, step 1).
gRPC	`Detokenize`	Resolve a token to its PAN; callable *only* by the Network Gateway inside the CDE (S-06).

Data owned. Token Vault (DS-01): card tokens and encrypted PANs, accessed strictly by key (token → PAN).

Dependencies. None toward application services; it is called by the Buyer client (S-01) and the Network Gateway (S-06). It must be strongly consistent so a token is resolvable immediately within the same checkout.

Scaling characteristics. Stateless workers over an encrypted key-value store; scales on tokenization rate with a sub-25 ms P95 (S-01). Capacity and key management stay inside the CDE, which is isolated so it can be audited and scaled independently of the rest of the platform.

4.6 Network Gateway

Responsibility. The sole egress toward the Banking and Payment Networks. It submits authorization, capture, refund, and payout requests, and surfaces approvals, declines, and timeout/error signals. It resolves tokens through the Tokenization Service (S-06) so that only this component, inside the CDE, ever handles the PAN. It does *not* own payment or ledger state.

Representative APIs (internal, over gRPC inside the CDE).

RPC	Purpose
`Authorize`, `QueryStatus`	Authorize a payment and resolve a pending one (S-05).
`Refund`, `SubmitDisputeResponse`	Execute refund and dispute saga steps (S-12).
`InitiatePayout`	Execute a payout saga step (S-15).
`OpenDispute`	Receive a chargeback from the network and forward it (S-13).

Data owned. None. It is stateless and resolves tokens in the Token Vault through the Tokenization Service; the external link uses mutual TLS over a private channel (ISO 8583 / ISO 20022).

Dependencies. Tokenization Service (S-06), and the external Banking and Payment Networks. Called by the Payment Orchestrator (S-05), the Refund and Dispute Service (S-12), and the Settlement and Reconciliation Service (S-15).

Scaling characteristics. Stateless; scales on authorization throughput. Because the external network latency dominates and is outside the project's control (NFR-05 excludes it), the gateway enforces per-call deadlines; a missing network response makes the payment `pending` rather than `failed` (ADR-3, UC-01 alt. 4b).

4.7 Payment Orchestrator

Responsibility. Coordinates the synchronous critical path of payment authorization (UC-01) and owns the payment lifecycle. It enforces idempotency, validates state transitions against an explicit state machine, calls fraud scoring and network authorization, and commits the payment state and its outbox record in one transaction. It decides the payment's lifecycle transition; it does *not* score risk itself, post journals, or deliver notifications.

The service enforces idempotency with a client-provided `Idempotency-Key` scoped to `(seller_id, key)`. A repeated key with equivalent parameters returns the stored response and runs nothing twice; a key reused with conflicting parameters is rejected (ADR-3).

Representative APIs.

Method	Path	Purpose
POST	`/v1/payments`	Create and authorize a payment; requires `Idempotency-Key` (FR-01, FR-02; S-02).
GET	`/v1/payments/{id}`	Read-your-writes status of a recently changed payment (FR-03; S-07).
POST	`/v1/payments/{id}/review`	Approve or reject a suspicious payment (UC-03; S-09).
POST	`/v1/payments/{id}/cancel`	Cancel a payment when the state machine permits it.

Responses carry a stable `payment_id` and a well-defined status drawn from the lifecycle `created` → `processi` with side states `requires_review`, `pending`, `failed`, `refunded`, `partially_refunded`, and `disputed`.

Data owned. Payment Store with outbox (DS-02): payments, payment methods, attempts, and status transitions; and the Idempotency Store (DS-03): key scope, request fingerprint, stored response, with a 24-hour time to live. It publishes `payment.*` events (A-01, A-02).

Dependencies. Edge Gateway (S-02), Fraud Scoring Service (S-04), Network Gateway (S-05); downstream consumers receive its events asynchronously (A-01). External networks are reached only through the Network Gateway.

Scaling characteristics. Stateless application instances over a distributed relational store sharded by `seller_id` and quorum-replicated across three availability zones, giving RPO = 0 (NFR-07, ADR-4). Scaling is driven by request volume, idempotency lookups, and database write contention. A slow fraud or network dependency can pile up in-flight requests, so bounded deadlines, concurrency limits, circuit breakers, and backpressure are mandatory (ADR-3). The success response is returned only after the authoritative payment state is durably committed; the P95 budget of the whole path is about 220 ms, within the 500 ms limit of NFR-05.

4.8 Fraud Scoring Service

Responsibility. Evaluates payment risk from transaction attributes, velocity features, configurable rules, and a machine-learning model, and returns a decision (`allow`, `review`, or `block`) within a 100 ms deadline (UC-03, FR-04, NFR-05). This target is consistent with Stripe's reported ability to reach a fraud decision in under 100 milliseconds [**?**, **?**]. It provides a risk decision; it does *not* authorize payments or post ledger entries.

Representative APIs.

RPC / Method	Purpose
`ScorePayment` (gRPC)	Return a risk score and decision for a payment (S-04).

RPC / Method	Purpose
GET `/v1/fraud/assessments/{id}`	Retrieve a stored assessment (score, decision, `rules_fired`, `model_version`).

Requests carry a stable `payment_id` and the minimum attributes needed for scoring; responses include the decision, an `assessment_id`, the model/ruleset version, and whether the local-rules fallback was used.

Data owned. Fraud Feature Store (DS-05): online velocity features (key-value) and offline analysis data (columnar), plus assessments. It publishes `fraud.scored` (A-03) and consumes payment and dispute events as feedback (A-01, A-06). Model-training pipelines live in a separate data platform outside the payment-processing boundary.

Dependencies. Called synchronously by the Payment Orchestrator (S-04); consumes feedback events asynchronously. Online features must be readable in a few milliseconds to fit the 100 ms budget.

Scaling characteristics. Stateless inference workers scaling on request rate, inference latency, and CPU/GPU utilization, with model artifacts cached in memory. Because fraud scoring is on the critical path, the architecture defines a strict deadline and an explicit failure policy: when the circuit opens, a *local-rules* fallback keeps the path alive, and high-value transactions move to `requires_review` rather than proceeding unscored (ADR-3, UC-03 alt. 3a). Allowing a payment to proceed without evaluation is never an automatic fallback.

4.9 Ledger Service

Responsibility. The authoritative financial record. It posts balanced double-entry journals from domain events, preserves accounting invariants, maintains balances, and reserves or releases funds for the refund and payout sagas (FR-07, ADR-4). Entries are immutable after posting; corrections are reversing journals, never in-place edits.

Representative APIs.

Method / RPC	Operation	Purpose
gRPC	`ReserveFunds`, `ReleaseReservation`	Reserve/release funds as a saga step (S-11, S-14).
GET	`/v1/ledger/accounts/{id}/balance`	Retrieve an authoritative balance.
GET	`/v1/ledger/journals/{id}`	Retrieve a posted journal and its entries.

Posting is event-driven: a journal carries a unique `source_event_id`; a replayed event is rejected, and debits must equal credits per currency. Journals are not created through a public write API; they are posted from `payment.*`, `refund.*`, and `payout.*` events.

Data owned. Ledger with outbox (DS-04): accounts, journals, entries, balances, and reservations. It publishes `ledger.journal_posted` (A-04).

Dependencies. Consumes events from the Payment Orchestrator, Refund and Dispute Service, and Settlement and Reconciliation Service (A-01, A-05, A-07); called synchronously for fund reservations by the Refund and Dispute Service (S-11) and Settlement and Reconciliation Service (S-14).

Scaling characteristics. The most conservative strategy, because correctness takes precedence over raw write throughput. Stateless API instances scale horizontally, but the store is distributed relational, append-only, serializable per account, and sharded by `seller_id` so that a journal's multi-row write stays within one shard without cross-shard transactions (ADR-4). Read replicas and event-fed read models serve reporting but are never presented as authoritative when they can be stale. Very large sellers need sub-partitioning and hot-account sub-accounts. The platform's peak workload (NFR-03: $\approx$ 98.6 million requests per hour) must be validated against the real transaction model and shard sizing by load testing; it is not guaranteed by adding instances.

4.10 Refund and Dispute Service

Responsibility. Orchestrates the long-running refund (UC-04, FR-05) and dispute (UC-07, FR-08, FR-09) lifecycles as sagas with persisted state and compensating actions (ADR-6). It drives each step, stores evidence, tracks deadlines, and reserves funds through the Ledger before refunding through the network. It does *not* post journals directly or hold card data.

Representative APIs.

Method	Path	Purpose
POST	`/v1/refunds`	Request a refund for an eligible payment; returns `pending` (FR-05; S-10).
GET	`/v1/disputes`, `/v1/disputes/{id}`	List and view disputes (FR-08).
POST	`/v1/disputes/{id}/responses`	Submit a dispute response and evidence (FR-09; S-10).

The network can also open a chargeback inbound through the Network Gateway (`OpenDispute`, S-13), which is acknowledged only after the dispute is durably stored.

Data owned. Refund and Dispute Store with outbox (DS-07): refunds, disputes, evidence references (by content hash), and saga state. It publishes `refund.*` and `dispute.*` events (A-05, A-06).

Dependencies. Ledger Service (S-11, reserve/release funds), Network Gateway (S-12), object storage for evidence; it consumes and publishes events through the Event Log.

Scaling characteristics. Stateless orchestrator over a relational store, scaling on saga volume and the number of in-flight disputes (which can last days). Every saga step is idempotent and retried with the same key; on network rejection, a compensating reversing journal releases the reservation (ADR-6). The orchestrator is a critical component: its failure pauses in-flight sagas until it recovers, so its state must be backed up with the store.

4.11 Settlement and Reconciliation Service

Responsibility. Processes payouts and reconciles internal ledger records against the statement files delivered by the banking networks, flagging missing, duplicated, or mismatched records until resolution (UC-06, UC-10, FR-07, FR-13, FR-14). Settlement moves funds; reconciliation verifies that records agree. In the prototype, external statements may be simulated files.

Representative APIs.

Method	Path	Purpose
POST	`/v1/reconciliation-runs`	Start a reconciliation run (FR-13; S-17).
GET	`/v1/reconciliation-runs/{id}`	Retrieve run status and discrepancies (FR-14).
GET	`/v1/settlements/{id}`	Retrieve payout/settlement status.

A run is accepted synchronously (returns quickly) and processed in the background (S-17).

Data owned. Settlement Store with outbox (DS-06): statements, statement lines, payouts, reconciliation runs, and discrepancies; raw statement files are kept unchanged as objects for evidence. It publishes `payout.*` and `reconciliation.*` events (A-07, A-08).

Dependencies. Ledger Service (S-14, reserve/release for payouts; `journal_posted` events for reconciliation input, A-04), Network Gateway (S-15), object storage, and the batch file feed from the networks (B-01, SFTP). It never reads the ledger store directly.

Scaling characteristics. Scales by processing independent batches, date ranges, or sellers in parallel; throughput depends on file size, matching complexity, and database queries. Jobs are restartable and idempotent, identified by file hash, so reprocessing never creates duplicate postings. Long-running work runs asynchronously with checkpointing so it does not interfere with the latency-sensitive payment path.

4.12 Monitoring and Audit Service

Responsibility. Keeps a tamper-evident audit trail and operational telemetry for investigation, accountability, and compliance evidence, and serves the Administrator operations view (UC-08, UC-10, FR-10). Audit evidence is distinguished from ordinary logs and protected against alteration; records are hash-chained. It does *not* own domain state; source services remain responsible for emitting accurate events.

Representative APIs.

Method / interface	Path	Purpose
(internal)	audit ingestion (events, telemetry)	Receive structured audit events and metrics (A-13).
GET	`/v1/operations/status`	Current system/operational status for Administrators (FR-10; S-18).
GET	`/v1/audit-records`, `/v1/audit-records/{id}`	Search and retrieve audit records for authorized Administrators and Auditors (S-18).

Data owned. Audit Store (DS-10), append-only write-once (WORM), hash-chained; and Telemetry Store (DS-13), time-series. It consumes events from every service (A-01–A-10) and publishes none that move money.

Dependencies. Consumes domain events and telemetry from all services through the Event Log (A-13). Depends on identity and access management for read authorization.

Scaling characteristics. Ingestion scales horizontally through buffered, batched writes; search is separated from ingestion so investigations do not degrade intake. Storage grows with event volume, record size, and retention. Audit records must never change once written, and telemetry is best-effort (at-most-once, A-13): losing a metric sample does not affect correctness. The service avoids storing cardholder data or secrets in audit payloads (NFR-11).

4.13 Notification and Webhook Dispatcher

Responsibility. Delivers subscribed events to merchant HTTPS endpoints as signed webhooks and sends notifications through external providers (UC-05, FR-06). Delivery is asynchronous and must never block the synchronous financial path (ADR-2). The role the proposal assigned to a separate "event publishing service" is *not* a distinct service here: reliable publication is achieved by the transactional outbox in each owning service plus the shared Event Log (DS-11, ADR-2); this dispatcher is the consumer that performs external delivery.

Representative APIs.

Method	Path	Purpose
POST	`/v1/webhook-endpoints`	Register a webhook endpoint (FR-06; S-19).
GET	`/v1/webhook-endpoints`, `/v1/webhook-deliveries/{id}`	List endpoints and inspect delivery status.
POST	`/v1/webhook-deliveries/{id}/retry`	Request an authorized manual retry.
(internal)	event consumer	Consume domain events to dispatch (A-11, A-12).

Data owned. Notification and Webhook Store (DS-09): endpoint configuration, subscription rules, delivery records, attempt counts, next-retry timestamps, and signing-key references. It publishes `notification.delivery_status` (A-10).

Dependencies. Consumes events from the Payment Orchestrator, Refund and Dispute, Settlement, Ledger, and Authentication services (A-11); depends on durable work queues with dead-letter queues, the Seller's System, external notification providers (A-12), and secrets management for webhook signing.

Scaling characteristics. Delivery workers scale on queue depth, arrival rate, retry backlog, and endpoint latency, with a concurrency limit *per destination* so one failing merchant endpoint cannot consume all workers. Delivery is at-least-once with exponential backoff and jitter: webhooks are retried for up to three days and then dead-lettered, and are signed with HMAC SHA-256 so merchants can verify authenticity and deduplicate by `event_id` [**?**]. This matches the 72-hour retry window stated in Section 6.5 (A-11). Exactly-once execution at the merchant endpoint is not guaranteed.

4.14 Read Models and Query Service

Responsibility. Serves dashboards, transaction search, balances, and reports built from events, under Command Query Responsibility Segregation (ADR-5). It answers read-heavy queries without competing with the payment write path (UC-02, UC-06, FR-03, FR-07, FR-08). It never writes to another service's store and is never a source of truth for money-moving decisions.

Representative APIs.

Method	Path	Purpose
GET	`/v1/payments`	Search payments (FR-03; S-08).
GET	`/v1/balance`	Seller balance and settlement dashboard (FR-07; S-08).
GET	`/v1/settlements`, `/v1/disputes`	Settlement and dispute views (FR-08; S-08).

A request for an object changed in the last few seconds is served from its owner (for example, the Payment Orchestrator, S-07) to give read-your-writes behavior.

Data owned. Read Model Store (DS-12): query projections (columnar and search index), rebuilt by replaying events.

Dependencies. Consumes events from all producing services through the Event Log; depends on no other store for writes.

Scaling characteristics. Stateless query tier plus independently scalable projections, scaling on query rate and index size. Projections lag the write model with a target under 5 s (eventual consistency, ADR-5); dashboards must accept and document this. A search outage degrades dashboards but never stops payments.

4.15 Cross-Service Data Ownership and Consistency

The catalog obeys the ownership and consistency rules established in Section 5:

1. **Payment state.** The Payment Orchestrator is authoritative for the payment lifecycle and idempotency (DS-02, DS-03).
2. **Financial state.** The Ledger Service is authoritative for posted journals (DS-04). A payment status is never treated as proof of a completed accounting operation.
3. **Fraud decisions.** The Fraud Scoring Service owns assessments (DS-05); the Payment Orchestrator decides the resulting lifecycle transition.
4. **Card data.** The Tokenization Service owns tokens and PANs (DS-01), confined to the CDE; only the Network Gateway may detokenize.
5. **Event publication.** A state change and its event are written atomically through the transactional outbox; the Event Log (DS-11) is shared infrastructure, never the system of record.
6. **Webhook delivery.** The Notification and Webhook Dispatcher owns delivery status (DS-09), not the underlying payment state.
7. **Settlement and reconciliation.** The Settlement and Reconciliation Service owns runs and discrepancies (DS-06); financial corrections follow the authorized ledger-posting process.
8. **Audit records.** The Monitoring and Audit Service owns the immutable audit trail (DS-10); source services remain responsible for emitting accurate events.

No operation requires a single distributed ACID transaction across services. Strong consistency is enforced within each authoritative financial operation, while cross-service workflows use durable events, idempotent consumers, retries, and reconciliation (ADR-2, ADR-6). These mechanisms must be tested under duplicate requests, concurrent operations, delayed events, partial failures, and recovery after service or database outages.

4.16 Scaling and Capacity Considerations

The catalog supports independent capacity planning, but the platform workload must be interpreted carefully: a single payment generates several API requests, a fraud evaluation, multiple database operations, ledger entries, and downstream events, so *transaction* throughput is not the same as *request* or *message* throughput. The reference figures are the ones fixed in Delivery 1: ≈ 20.8 million requests per hour on average (WL-01, NFR-02) and ≈ 98.6 million at peak (WL-02, NFR-03), a 35/65 read/write split (WL-04), a P95 of 500 ms for synchronous requests with a 100 ms fraud decision (NFR-05), and RPO = 0 for acknowledged financial state (NFR-07).

For each service, capacity planning estimates incoming requests or events per second, average and peak latency, CPU and memory per operation, database reads/writes and contention, message production/consumption rates, retry amplification during failures, redundancy and failover capacity, and the requirements under the 20× growth scenario reserved for Delivery 3. Stateless services (Edge Gateway, Authentication, Tokenization, Payment Orchestrator tier, Read Models) are the simplest to scale horizontally; messaging and background services (Notification and Webhook Dispatcher, Settlement and Reconciliation, the Event Log) require careful partitioning and backlog management; and

the Ledger Service and the other authoritative financial stores require workload-specific benchmarks and explicit consistency guarantees before their capacity can be considered sufficient (ADR-4). The final architecture must therefore show not only that each service scales independently, but that the end-to-end payment workflow stays correct, observable, and recoverable under peak load and partial failure.

5 Data Architecture

This section defines, for each service of Section 3, how its data is stored, modeled, kept consistent, and protected by transactions. It is cloud-independent: it names storage types, not products. Data stores keep the identifiers used in the architecture diagram (DS-01 to DS-13).

5.1 Data Ownership Model

Every piece of data has exactly one owning service, and only that service writes to its data store. Other services obtain the data in one of three ways, and never by reading or writing another service's store directly:

- **API call**, when the caller needs the current value immediately (for example, the Edge Gateway validating an access token with the Authentication Service).
- **Event subscription**, when the caller can work with data that arrives a few seconds later (for example, the Ledger Service posting journals from payment events).
- **Read model**, when the data is needed for queries, dashboards, or reports. Read models are built by the Read Models and Query Service from events, never by replicating another service's database.

Table 17: Data ownership and cross-service access.

Service	**Data owned (store)**	**How other services access it**
Authentication Service	Users, roles, MFA factors, sellers and their verification data (DS-08)	API: token validation, seller verification. Events: `seller.verification_updated`. Other services keep only the opaque `seller_id`.
Tokenization Service	Card tokens and encrypted PANs (DS-01)	API: `Tokenize` (clients), `Detokenize` (only the Network Gateway, inside the CDE).
Payment Orchestrator	Payments, payment methods, attempts, status transitions (DS-02); idempotency records (DS-03)	API: payment creation and status of recent payments. Events: `payment.*`.

Table 17 (continued)

Service	Data owned (store)	How other services access it
Fraud Scoring Service	Risk features and assessments (DS-05)	API: `ScorePayment`. Events: `fraud.scored`.
Ledger Service	Ledger accounts, journals, entries, balances, reservations (DS-04)	API: `ReserveFunds`, `ReleaseReservation`. Events: `ledger.journal_posted`.
Settlement and Reconciliation Service	Statements, payouts, reconciliation runs, discrepancies (DS-06)	API: reconciliation runs. Events: `payout.*`, `reconciliation.*`.
Refund and Dispute Service	Refunds, disputes, evidence, saga state (DS-07)	API: refunds and dispute responses. Events: `refund.*`, `dispute.*`.
Monitoring and Audit Service	Audit records (DS-10); telemetry (DS-13)	API: read-only queries for Administrators and Auditors.
Notification and Webhook Dispatcher	Webhook endpoints, deliveries, notification status (DS-09)	API: webhook endpoint management. Events: `notification.delivery_status`.
Read Models and Query Service	Query projections (DS-12)	API: search, dashboards, reports. It never writes to any other store.
Edge Gateway, Network Gateway	None (stateless)	Not applicable.

The Event Log (DS-11) is shared *infrastructure*, not a shared database: each event type has a single producing service, which also owns its schema, and the log is never the system of record. The source of truth always remains in the producer's own store.

5.2 Storage Strategy per Service

Each service uses the storage type that matches its access pattern (Table 18). Relational storage is used where the service needs multi-row ACID transactions and integrity constraints on money; key-value, time-series, columnar, and object storage are used where access is by key, by time, by aggregation, or by whole file.

Table 18: Storage type per service and its justification.

Service (store)	Storage type	Why
Payment Orchestrator (DS-02)	Distributed relational, sharded by `seller_id`, consensus-replicated	A payment, its attempt, its status transition, and its outbox record must be written atomically, and only valid state transitions may be accepted (FR-02). Majority replication across three availability zones gives RPO = 0 (NFR-07).
Payment Orchestrator (DS-03)	Key-value with time to live	Each lookup is a single key; an atomic insert-if-absent prevents double charges, and records expire automatically after 24 hours.
Tokenization Service (DS-01)	Encrypted key-value	Access is always token → PAN by key, with no relations; a small, simple store keeps the CDE minimal (NFR-11).
Fraud Scoring Service (DS-05)	Key-value (online features) and columnar (offline analysis)	Velocity counters must be read in a few milliseconds to fit the 100 ms decision (NFR-05); rule and model analysis runs aggregations over large histories.
Ledger Service (DS-04)	Distributed relational, append-only	A journal writes several entries that must sum to zero in one transaction, and a unique constraint on the source event prevents double posting (FR-07).
Settlement and Reconciliation Service (DS-06)	Relational and object storage	Reconciliation joins statement lines with ledger references (FR-13, FR-14); raw statement files are kept unchanged as objects for evidence.
Refund and Dispute Service (DS-07)	Relational and object storage	Refund and dispute state machines need ACID updates (FR-05, FR-09); evidence documents are stored as objects referenced by hash.
Authentication Service (DS-08)	Relational and encrypted object storage	Users, roles, and sellers are related and need uniqueness constraints (FR-11, FR-12); verification documents are files.
Monitoring and Audit Service (DS-10, DS-13)	Append-only write-once storage (audit); time-series (telemetry)	Audit records must never change and are hash-chained (UC-10); metrics are written and queried by time window (FR-10).
Notification and Webhook Dispatcher (DS-09)	Key-value and durable queue	Each delivery is read and updated by its identifier; the queue holds pending retries (FR-06).
Read Models and Query Service (DS-12)	Columnar and search index	Dashboards aggregate many rows and transaction search filters by many fields (FR-03, FR-07); neither fits the write-optimized transactional stores.

Figure 3 places these stores in their trust zones and shows how data moves from the transactional stores to the derived stores through the Event Log.

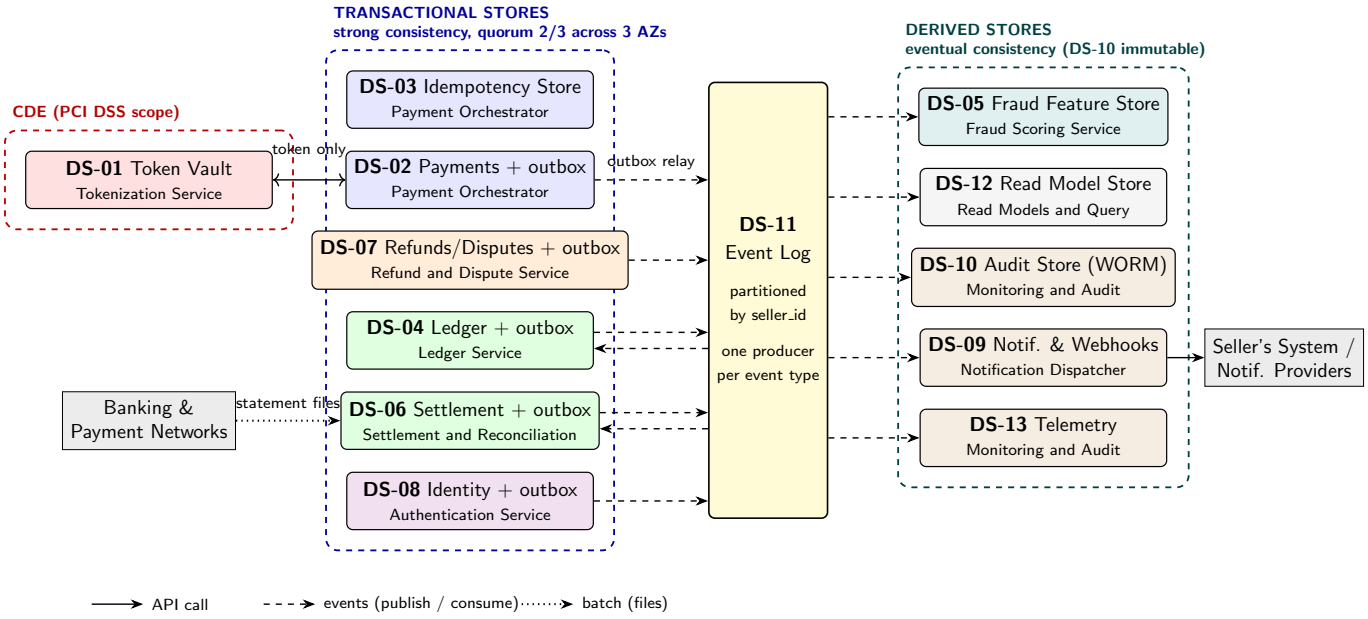


Figure 3: Data stores by trust zone and owning service, and data flow through the Event Log.

5.3 Data Model

Figure 4 shows the key entities of every service and their relationships. Entities are grouped by owning service. A foreign key (FK) is enforced only inside one store; when an entity refers to data owned by another service, it keeps the identifier as a reference (REF), which is never enforced by a database constraint and is resolved through that service's API or events. This is how the model avoids a shared database.

Payment state machine. A payment moves through `created` → `processing` → `authorized` → `captured`, with the side states `requires_review` (UC-03), `pending` (no network response, UC-01 alternative 4b), `failed`, `refunded` or `partially_refunded` (UC-04), and `disputed` (UC-07). Only these transitions are accepted, and each one is recorded as a `StatusTransition`.

Double-entry ledger. Every money movement is a `Journal` with at least two `LedgerEntry` rows whose debits and credits sum to zero per currency. Entries are never updated; errors are corrected with reversing journals. Amounts are 64-bit integers in the currency's minor unit with an ISO 4217 code, never floating-point values. For example, a captured payment of USD 100.00 with a fee of 2.9% + USD 0.30 produces one journal:

Ledger account	**Debit**	**Credit**
Network receivable	10,000	
Seller pending balance		9,680
Platform fee revenue		320
Total (minor units)	**10,000**	**10,000**

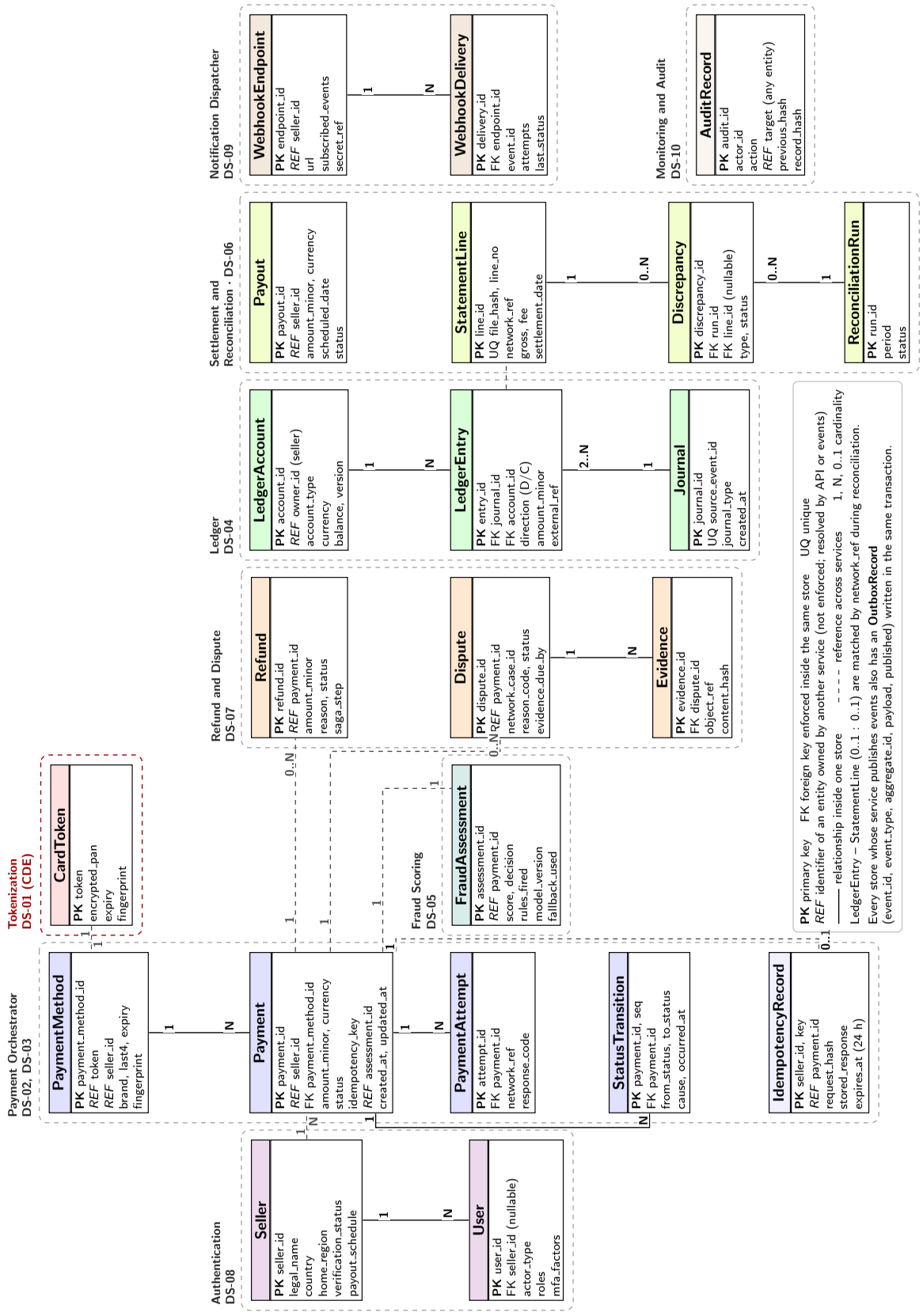


Figure 4: Entity-relationship diagram of the platform, grouped by owning service.

5.4 Consistency Requirements

Strong consistency is used wherever a stale read could move money incorrectly or grant access that should be denied. Eventual consistency is used for derived data, where availability and latency matter more. Causal consistency applies to everything built from events: because events are partitioned by `seller_id`, every consumer sees the changes of one seller in the order they happened.

Table 19: Consistency requirement per service.

Service (store)	**Consistency**	**Why**
Payment Orchestrator (DS-02, DS-03)	Strong (linearizable per payment and per idempotency key)	A stale status could authorize a payment twice or report a wrong result to the Buyer (FR-02, FR-03).
Tokenization Service (DS-01)	Strong	A token must be resolvable immediately after it is created, during the same checkout.
Ledger Service (DS-04)	Strong, serializable per account; causal for incoming events	Balances must be exact (FR-07); events of one seller must be posted in order.
Settlement and Reconciliation Service (DS-06)	Strong for payouts and runs; causal for incoming ledger events	A payout must never be created twice; reconciliation needs the seller's journals in order (FR-13, FR-14).
Refund and Dispute Service (DS-07)	Strong per refund or dispute	A refund must never be issued twice, and dispute deadlines must be exact (FR-05, FR-09).
Authentication Service (DS-08)	Strong	A revoked role or a rejected seller must take effect at once (FR-11, FR-12).
Fraud Scoring Service (DS-05)	Eventual (seconds)	Velocity counters that are a few seconds old do not change the decision materially; latency matters more (NFR-05).
Notification and Webhook Dispatcher (DS-09)	Strong per delivery record; eventual with respect to the source event	Each delivery attempt must be counted correctly, but a notification may arrive seconds after the event (FR-06).
Monitoring and Audit Service (DS-10, DS-13)	Audit: immutable once written, eventual with respect to the action. Telemetry: eventual	An audit record may arrive seconds later but can never change (UC-10); metrics tolerate small delays (FR-10).

Table 19 (continued)

Service (store)	Consistency	Why
Read Models and Query Service (DS-12)	Eventual (target lag < 5 s)	Dashboards and search do not move money. A payment changed in the last few seconds is read from the Payment Orchestrator instead, which gives read-your-writes behavior (UC-02).

5.5 Transaction Boundaries

Each ACID transaction is local to one service and one store. Because every financial row carries `seller_id` and the financial stores are sharded by it, all rows written by a transaction belong to the same shard, so no transaction ever spans shards. Table 20 lists the operations that require ACID transactions and how work that crosses services is coordinated.

Table 20: ACID transaction boundaries and cross-service coordination.

Operation	Service	Local ACID transaction (atomic unit)	Across services
Authorize payment (UC-01)	Payment Orchestrator	Payment + attempt + status transition + outbox record, in DS-02. The idempotency key is inserted in DS-03 before processing (insert-if-absent). No transaction stays open during the fraud or network calls.	Fraud scoring and network authorization are synchronous calls made before the commit. Ledger, notifications, audit, and read models follow through events.
Post journal	Ledger Service	Journal + its entries (sum zero) + balance updates + outbox record, serializable per account. The unique `source_event_id` rejects a duplicate event.	Triggered by events; no coordination needed.
Reserve or release funds	Ledger Service	Reservation journal and balance check in one transaction; it fails if the available balance is insufficient.	Called synchronously as a step of the refund and payout sagas.
Refund (UC-04)	Refund and Dispute Service	Each saga step: refund state + saga step + outbox record.	Orchestrated saga: create refund → reserve funds → network refund → `refund.succeeded` (Ledger posts the final journal) or, on rejection, release the reservation (compensation).

Table 20 (continued)

Operation	Service	Local ACID transaction (atomic unit)	Across services
Dispute response (UC-07)	Refund and Dispute Service	Dispute status + evidence metadata + outbox record; the file is stored first and referenced by hash.	Submission to the network is a saga step, retried with the same idempotency key.
Payout	Settlement and Reconciliation Service	Payout state + outbox record.	Orchestrated saga: create payout → reserve funds → network payout → `payout.paid` or release the reservation.
Statement ingestion and reconciliation (UC-10)	Settlement and Reconciliation Service	Statement lines + unique file hash, per file; discrepancies + run status + outbox record, per batch.	Ledger data arrives through `ledger.journal_posted` events; the ledger store is never read directly.
Seller verification (UC-09)	Authentication Service	Seller verification status + outbox record.	Notification and audit follow through events.

Distributed transactions. The platform does not use distributed ACID transactions (two-phase commit) across services. The Banking and Payment Networks cannot take part in one, and locks held across services during network calls would harm availability (NFR-06) and latency (NFR-05). Instead, three mechanisms keep multi-service work correct: the *transactional outbox*, which writes a state change and its event atomically (Architecture Decision Record 2); *orchestrated sagas* with compensating actions for refunds, disputes, and payouts (Architecture Decision Record 6); and *idempotent consumers*, which make at-least-once delivery safe.

5.6 Shared-Data Anti-Patterns and How They Are Avoided

Table 21: Shared-data anti-patterns avoided by the design.

Anti-pattern	How the design avoids it
Shared database between services	Each store has one owner (Table 17) and its own credentials; no other service can connect to it.
Reading another service's tables for reports or reconciliation	Queries use read models built from events (Architecture Decision Record 5); reconciliation consumes `ledger.journal_posted` events instead of reading the ledger.
Direct access to the Token Vault by other CDE components	The Network Gateway resolves tokens through the Tokenization Service's `Detokenize` operation, not by reading DS-01.

Table 21 (continued)

Anti-pattern	How the design avoids it
Distributed transactions across services	Local ACID transactions plus sagas and the outbox (Section 5.5).
Dual write (store, then publish separately)	Transactional outbox: the state change and its event are committed together.
Editing copies of another service's data	Consumers keep read-only projections; corrections are made only by the owner and propagate as new events.
Using the Event Log as a database	Events are retained for 7 days for replay; the system of record is always the producer's store.

6 Communication Architecture

This section defines the communication pattern of every significant interaction between the services of Section 3, and justifies each choice against the requirements of Delivery 1. It is cloud-independent: it names protocols and patterns, not products.

6.1 Selection Criteria and Protocols

An interaction is **synchronous** only when the caller needs the answer to continue, the operation is short, and a failure must be reported to the caller at once. Every other interaction is **asynchronous**: the caller does not need an immediate answer, the work is long-running or not critical for the current request, or the producer must be decoupled from its consumers to absorb peaks (NFR-03, NFR-04). The protocols are:

- **REST over HTTPS (TLS 1.3)** for requests from Buyers, Sellers, Administrators, and Auditors, which enter through the Edge Gateway.
- **gRPC over HTTP/2 with mutual TLS** for synchronous calls between services, because it supports typed contracts and deadline propagation.
- **Event Log (publish/subscribe)** for domain events, partitioned by `seller_id` so that the events of one seller are consumed in order.
- **Durable work queues** with dead-letter queues for webhook and notification deliveries.
- **HTTPS webhooks** signed with HMAC SHA-256 toward the Seller's System, and a **private mutual-TLS link** (ISO 8583 / ISO 20022) toward the Banking and Payment Networks, which also deliver settlement **files** by SFTP.

6.2 Communication Diagram

Figure 5 shows the main paths. Steps 1 to 7 correspond to the payment flow of UC-01: tokenization (1), payment request (2), idempotency check (3), fraud decision (4), network authorization (5), commit of the payment and its outbox record (6), and response to the client (7).

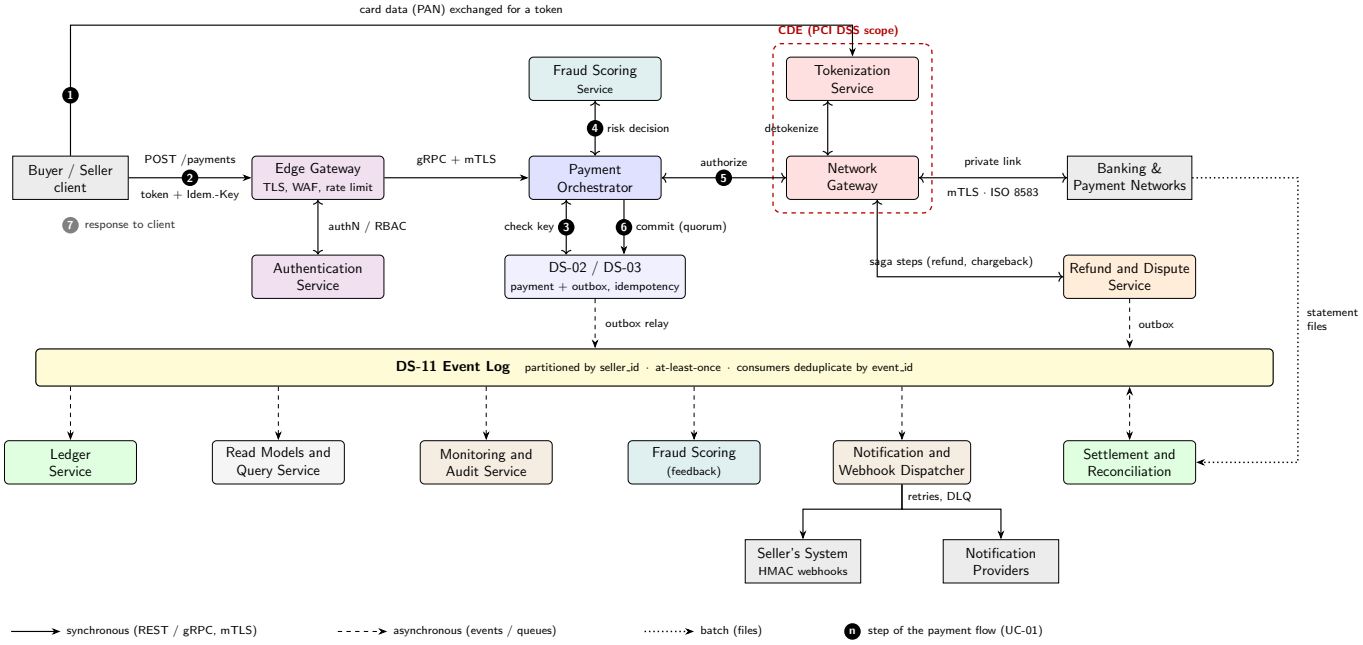


Figure 5: Communication diagram: main synchronous paths of the payment flow (steps 1–7), asynchronous event propagation, and batch settlement exchange. Table 22 lists every interaction.

6.3 Communication Matrix

Table 22 lists every inter-service interaction with its pattern and protocol. Requests from clients pass through the Edge Gateway, which validates the access token (S-03) before forwarding them.

Table 22: Communication matrix: all interactions, pattern, and protocol.

ID	From	To	Pattern	Protocol
S-01	Buyer client	Tokenization Service	Sync	REST / HTTPS
S-02	Buyer client (via Edge Gateway)	Payment Orchestrator	Sync	REST, then gRPC
S-03	Edge Gateway	Authentication Service	Sync	gRPC
S-04	Payment Orchestrator	Fraud Scoring Service	Sync	gRPC
S-05	Payment Orchestrator	Network Gateway	Sync	gRPC
S-06	Network Gateway	Tokenization Service	Sync	gRPC (inside CDE)
S-07	Buyer or Seller client	Payment Orchestrator	Sync	REST, then gRPC

Table 22 (continued)

ID	From	To	Pattern	Protocol
S-08	Seller, Administrator, or Auditor client	Read Models and Query Service	Sync	REST, then gRPC
S-09	Administrator client	Payment Orchestrator	Sync	REST, then gRPC
S-10	Seller client	Refund and Dispute Service	Sync	REST, then gRPC
S-11	Refund and Dispute Service	Ledger Service	Sync	gRPC
S-12	Refund and Dispute Service	Network Gateway	Sync	gRPC
S-13	Network Gateway	Refund and Dispute Service	Sync	gRPC
S-14	Settlement and Reconciliation Service	Ledger Service	Sync	gRPC
S-15	Settlement and Reconciliation Service	Network Gateway	Sync	gRPC
S-16	Seller or Administrator client	Authentication Service	Sync	REST, then gRPC
S-17	Auditor client	Settlement and Reconciliation Service	Sync	REST, then gRPC
S-18	Administrator or Auditor client	Monitoring and Audit Service	Sync	REST, then gRPC
S-19	Seller client	Notification and Webhook Dispatcher	Sync	REST, then gRPC
A-01	Payment Orchestrator	Ledger, Dispatcher, Monitoring and Audit, Fraud Scoring, Read Models	Async	Event Log
A-02	Payment Orchestrator	Read Models, Dispatcher, Monitoring and Audit	Async	Event Log
A-03	Fraud Scoring Service	Monitoring and Audit, Read Models	Async	Event Log

Table 22 (continued)

ID	From	To	Pattern	Protocol
A-04	Ledger Service	Settlement and Reconciliation, Read Models, Monitoring and Audit	Async	Event Log
A-05	Refund and Dispute Service	Ledger, Dispatcher, Monitoring and Audit, Read Models	Async	Event Log
A-06	Refund and Dispute Service	Dispatcher, Fraud Scoring, Monitoring and Audit, Read Models	Async	Event Log
A-07	Settlement and Reconciliation Service	Ledger, Dispatcher, Monitoring and Audit, Read Models	Async	Event Log
A-08	Settlement and Reconciliation Service	Monitoring and Audit, Read Models	Async	Event Log
A-09	Authentication Service	Dispatcher, Monitoring and Audit, Read Models	Async	Event Log
A-10	Notification and Webhook Dispatcher	Monitoring and Audit	Async	Event Log
A-11	Notification and Webhook Dispatcher	Seller's System	Async	Work queue → HTTPS webhook
A-12	Notification and Webhook Dispatcher	External Notification Providers	Async	Work queue → HTTPS
A-13	All services	Monitoring and Audit Service (telemetry)	Async	Telemetry stream
B-01	Banking and Payment Networks	Settlement and Reconciliation Service	Async (batch)	SFTP files

6.4 Synchronous Interactions

Table 23 documents each synchronous interaction: caller, callee, operation, and latency expectation at the 95th percentile, excluding the latency of the external networks (NFR-05). Every call carries a deadline; when it expires, the caller applies the failure handling shown.

Table 23: Synchronous interactions.

ID	Caller	Callee	Operation	Latency (P95)	On failure
S-01	Buyer client	Tokenization Service	`POST /v1/tokens`: exchange card data for a token (step 1)	$<$ 25 ms	Error to the Buyer; no payment is created.
S-02	Buyer client	Payment Orchestrator	`POST /v1/payments`: create and authorize a payment (steps 2–7)	$<$ 500 ms end to end; estimated $\approx$ 220 ms	Error or `pending`; the client retries with the same idempotency key.
S-03	Edge Gateway	Authentication Service	`ValidateToken`: identity and roles of the caller	$<$ 10 ms (cached)	Request rejected (fail closed).
S-04	Payment Orchestrator	Fraud Scoring Service	`ScorePayment`: risk decision (step 4)	$<$ 100 ms (deadline)	Local rules policy; high amounts go to `requires_review` (UC-03, 3a).
S-05	Payment Orchestrator	Network Gateway	`Authorize`, `QueryStatus`: network authorization (step 5)	$<$ 20 ms internal	Timeout $\rightarrow$ `pending`; resolved later with `QueryStatus` (UC-01, 4b).
S-06	Network Gateway	Tokenization Service	`Detokenize`: obtain the PAN inside the CDE	$<$ 10 ms	Authorization fails and is retried with the same key.
S-07	Buyer or Seller client	Payment Orchestrator	`GET /v1/payments/{id}`: status of a recently changed payment (FR-03)	$<$ 100 ms	Error to the client.
S-08	Seller, Administrator, or Auditor client	Read Models and Query Service	`GET /v1/payments`, `/v1/balance`, `/v1/settlements`, `/v1/disputes`: search, balance, and dashboards (FR-07, FR-08)	$<$ 300 ms	Error to the client; payments keep working.
S-09	Administrator client	Payment Orchestrator	`POST /v1/payments/{id}/review`: approve or reject a suspicious payment (UC-03)	$<$ 200 ms	Error to the Administrator.

Table 23 (continued)

ID	Caller	Callee	Operation	Latency (P95)	On failure
S-10	Seller client	Refund and Dispute Service	`POST /v1/refunds`, `POST /v1/disputes/{id}/responses`: request a refund, respond to a dispute (FR-05, FR-09)	< 300 ms (returns `pending`)	Error; the client retries with the same idempotency key.
S-11	Refund and Dispute Service	Ledger Service	`ReserveFunds`, `ReleaseReservation`: saga step (refund)	< 50 ms	Saga step retried; no funds, refund rejected.
S-12	Refund and Dispute Service	Network Gateway	`Refund`, `SubmitDisputeResponse`: saga step	< 20 ms internal	Retried with the same key; on rejection, compensation.
S-13	Network Gateway	Refund and Dispute Service	`OpenDispute`: chargeback received from the network	< 100 ms	The network is acknowledged only after the dispute is stored; the network resends.
S-14	Settlement and Reconciliation Service	Ledger Service	`ReserveFunds`, `ReleaseReservation`: saga step (payout)	< 50 ms	Saga step retried.
S-15	Settlement and Reconciliation Service	Network Gateway	`InitiatePayout`: saga step	< 20 ms internal	Retried with the same key; on rejection, compensation.
S-16	Seller or Administrator client	Authentication Service	`POST /v1/sellers/{id}/verification`, `POST /v1/verifications/{id}/decision` (FR-11, FR-12)	< 300 ms	Error to the client.
S-17	Auditor client	Settlement and Reconciliation Service	`POST /v1/reconciliation-runs`, `GET /v1/reconciliation-runs/{id}` (FR-13, FR-14); the run itself is processed in the background	< 300 ms	Error to the Auditor.
S-18	Administrator or Auditor client	Monitoring and Audit Service	`GET /v1/operations/status`, `GET /v1/audit-records` (FR-10)	< 500 ms	Error to the client.
S-19	Seller client	Notification and Webhook Dispatcher	`POST /v1/webhook-endpoints`: register an endpoint (FR-06)	< 300 ms	Error to the Seller.

The synchronous path of a payment (S-02) adds up to about 220 ms: edge 20 ms, authentication 10 ms, tokenization 25 ms, idempotency check 10 ms, fraud decision 100 ms, quorum commit 30 ms, and internal hops 25 ms. This is below the 500 ms limit of NFR-05.

6.5 Asynchronous Interactions

Table 24 documents each asynchronous interaction: producer, event type, consumers, and delivery guarantee. Exactly-once delivery is not assumed anywhere. Delivery is at-least-once, and every consumer deduplicates by `event_id`, which makes processing effectively once. Events that move money are published through a transactional outbox, so an event is never lost after its state change is committed.

Table 24: Asynchronous interactions.

ID	Producer	Event type	Consumers	Delivery guarantee and idempotency
A-01	Payment Orchestrator	`payment.authorized`, `.captured`, `.failed`, `.pending`	Ledger, Dispatcher, Monitoring and Audit, Fraud Scoring, Read Models	At-least-once through the outbox. The Ledger rejects duplicates with its unique `source_event_id`.
A-02	Payment Orchestrator	`payment.requires_review`	Read Models (review queue), Dispatcher, Monitoring and Audit	At-least-once through the outbox; deduplication by `event_id`.
A-03	Fraud Scoring Service	`fraud.scored`	Monitoring and Audit, Read Models	At-least-once, published directly with retries; it does not move money, and the decision is also stored in the payment.
A-04	Ledger Service	`ledger.journal_posted`	Settlement and Reconciliation, Read Models, Monitoring and Audit	At-least-once through the outbox; ordered per seller.
A-05	Refund and Dispute Service	`refund.requested`, `.succeeded`, `.failed`	Ledger, Dispatcher, Monitoring and Audit, Read Models	At-least-once through the outbox; the Ledger rejects duplicates.
A-06	Refund and Dispute Service	`dispute.opened`, `.updated`, `.deadline_approaching`	Dispatcher, Fraud Scoring, Monitoring and Audit, Read Models	At-least-once through the outbox; deduplication by `event_id`.

Table 24 (continued)

ID	Producer	Event type	Consumers	Delivery guarantee and idempotency
A-07	Settlement and Reconciliation Service	`payout.scheduled`, `.paid`, `.failed`	Ledger, Dispatcher, Monitoring and Audit, Read Models	At-least-once through the outbox; the Ledger rejects duplicates.
A-08	Settlement and Reconciliation Service	`reconciliation.discrepancy_found`	Monitoring and Audit, Read Models	At-least-once through the outbox.
A-09	Authentication Service	`seller.verification_updated`	Dispatcher, Monitoring and Audit, Read Models	At-least-once through the outbox.
A-10	Notification and Webhook Dispatcher	`notification.delivery_status`	Monitoring and Audit	At-least-once, published directly with retries.
A-11	Notification and Webhook Dispatcher	Webhook delivery of any subscribed event	Seller's System	At-least-once; retries with exponential backoff for up to 72 h, then dead-letter queue. Signed with HMAC; the Seller deduplicates by `event_id`.
A-12	Notification and Webhook Dispatcher	Email, SMS, and push requests	External Notification Providers	At-least-once; failover to a secondary provider; dead-letter queue.
A-13	All services	Metrics, traces, logs	Monitoring and Audit Service	At-most-once (best effort); losing a sample does not affect correctness.
B-01	Banking and Payment Networks	Settlement statement files	Settlement and Reconciliation Service	Each file is ingested once, identified by its hash; a resent file is ignored.

6.6 Justification of the Asynchronous Choices

Table 25 answers, for each asynchronous interaction, what would break if it were synchronous.

Table 25: What would break if each asynchronous interaction were synchronous.

Interaction	What would break if it were synchronous
Ledger posting (A-01, A-05, A-07 → Ledger)	The payment response would wait for a commit in a second service, and the two writes would need a distributed transaction. A slow or unavailable ledger, or a busy hot account, would fail or delay payments and break the 500 ms limit (NFR-05) and the 99.99% availability (NFR-06).

Table 25 (continued)

Interaction	What would break if it were synchronous
Webhooks and notifications (A-01, A-05, A-06, A-07, A-09 → Dispatcher; A-11, A-12)	Checkout would wait for the slowest Seller endpoint or notification provider. These are external systems that are often slow or down, so one failing merchant endpoint would block payments for everyone (cascading failure).
Audit trail (all events → Monitoring and Audit)	Every operation would also wait for the audit write, adding latency to the critical path; an audit outage would stop payments.
Read models (all events → Read Models)	Each payment would update search indexes and dashboards before answering. Index updates are heavy, so the payment latency would grow with query load, and a search outage would stop payments.
Fraud feedback (A-01, A-06 → Fraud Scoring)	The fraud service would be called twice per payment, and the second call does not change the Buyer's answer; it would only consume latency budget.
Reconciliation input (A-04 → Settlement)	The Ledger would depend on the availability of the Settlement service, although reconciliation works on whole periods and does not need each journal immediately.
Refund, dispute, and payout lifecycle (A-05, A-06, A-07)	These flows take from seconds (network refunds) to days (disputes). A synchronous request would hold connections and resources for that long, and client timeouts would cause repeated refund attempts. The Seller instead receives `pending` and is notified of the result.
Fraud and seller-verification records (A-03, A-09)	An Administrator's decision or a fraud score would wait for audit and notification writes that do not affect the result.
Delivery status (A-10)	The Dispatcher would block on the Monitoring and Audit Service, so a monitoring outage would stop notifications.
Telemetry (A-13)	An outage of the observability pipeline would take down the production services it is meant to observe.
Statement files (B-01)	Not possible: the networks deliver settlement only as periodic files.

These choices introduce eventual consistency, duplicates, and out-of-order delivery. They are handled as follows: derived data has a lag target below 5 s, and a payment changed in the last few seconds is read from its owner (S-07); every consumer deduplicates by `event_id`; events are ordered per `seller_id`; and messages that exhaust their retries go to a dead-letter queue, visible to the Administrator (UC-08) and replayable after correction.

7 Architecture Decision Records

Each record states one decision and follows the required structure: Decision, Context, Alternatives Considered, Rationale, Trade-offs and Consequences. Requirement references point to Delivery 1.

Record	Topic	Main requirements involved
1	Monolith versus microservices decomposition	Non-Functional Requirements 03, 04, 06, 11
2	Event-driven versus request-driven communication	Non-Functional Requirements 03, 05, 07; Functional Requirements 06, 07
3	Synchronous versus asynchronous payment processing	Use Case 01; Non-Functional Requirement 05
4	Relational versus non-relational storage	Non-Functional Requirement 07; Functional Requirements 02, 07, 13
5	Command Query Responsibility Segregation	Functional Requirements 03, 07, 13; Non-Functional Requirement 05
6	Saga versus distributed transactions	Use Cases 04, 07; Functional Requirements 05, 09

7.1 Architecture Decision Record 1: Coarse-grained services aligned with the Delivery 1 modules

Decision: The platform is decomposed into independently deployable services aligned with the six Delivery 1 modules, each owning its own data store, instead of one deployable application or one service per entity.

Context:

- Non-Functional Requirements 03 and 04 require 98.6 million requests per hour at peak, sustained for 4 hours.
- Non-Functional Requirement 06 requires 99.99 percent monthly availability, so one failing module must not stop the others.
- Non-Functional Requirement 11 requires a minimal scope under the Payment Card Industry Data Security Standard, so tokenization, the vault and the network gateway must be isolatable.
- Non-Functional Requirement 05 sets a 100-millisecond fraud decision, while reports and reconciliation have very different load profiles.

Alternatives Considered:

1. Modular monolith: a single deployable application with internal module boundaries.
2. Fine-grained microservices: one service per entity or use case (payment, attempt, refund, and so on).

Rationale: The module boundaries already exist in Delivery 1 and map cleanly to data ownership and use cases. Separate deployable units let the Cardholder Data Environment remain a small, auditable island. Independent scaling matches the workloads: fraud scoring is latency-bound, the read side is query-bound and settlement is batch-bound.

Trade-offs: The platform becomes a distributed system, with network hops, partial failures and a higher observability need. Internal calls consume part of the latency budget. There are no transactions across services. Operating costs are higher than for a monolith.

Consequences: Services communicate only through interfaces and events, never through shared databases. Contracts and event schemas must be versioned, and distributed tracing with a correlation identifier becomes mandatory. Moving a boundary later is costly, so the boundaries must be right now. A modular monolith remains a reasonable starting point for a small team, but it would not satisfy Non-Functional Requirements 11 and 06 without extra work.

7.2 Architecture Decision Record 2: Event-driven propagation after the payment commit, using a transactional outbox

Decision: After the payment state is committed, downstream work (ledger posting, notifications, read models, audit, fraud feedback) is driven by domain events published through a transactional outbox to a partitioned event log.

Context:

- Non-Functional Requirement 05 limits synchronous requests to 500 milliseconds at the 95th percentile.
- Non-Functional Requirement 07 requires a Recovery Point Objective of zero for acknowledged transactions.
- Non-Functional Requirements 03 and 04 require absorbing sustained peaks.
- Functional Requirement 06 requires reliable notifications and Functional Requirement 07 requires accurate balances.

Alternatives Considered:

1. Request-driven chain: the Payment Orchestrator synchronously calls the ledger, notifications and the read side.
2. Direct publish after commit (dual write): write to the database, then publish to a broker in a separate step.
3. Distributed transaction: two-phase commit across the payment store and the ledger.

Rationale: The outbox writes state and event in one local transaction, so a committed change always produces its event and an uncommitted change never does. The log decouples consumers, buffers peaks and supports replay. Using the seller identifier as partition key gives per-seller ordering, which the ledger needs.

Trade-offs: Downstream data is eventually consistent: ledger posting lags the capture by seconds (target under 5 seconds). Delivery is at-least-once, so duplicates occur. Operating the log has a cost, and events are harder to debug than direct calls.

Consequences: Every consumer must be idempotent and deduplicate by event identifier. Events must be versioned and carry a correlation identifier. A dead-letter queue and replay tooling are required. Adding a consumer becomes easy: it subscribes and can replay up to 7 days. Any use case that needs an immediate view must use a different read path (see Record 5).

7.3 Architecture Decision Record 3: Synchronous authorization with an explicit pending state

Decision: Payment authorization is processed synchronously, limited to the steps that determine its outcome, and an authorization without a network response is stored as pending instead of failed.

Context:

- Use Case 01 requires the Buyer to receive the result; the Buyer expects a fast, accurate status.
- Alternative flow 4b of Use Case 01 requires recording as pending any transaction that cannot complete immediately.
- Non-Functional Requirement 05 requires under 500 milliseconds at the 95th percentile and a fraud decision under 100 milliseconds, excluding network latency.
- Workload Parameter 06 sets a 95th-percentile target of 200 milliseconds.

Alternatives Considered:

1. Fully asynchronous: accept the request, answer "accepted", and deliver the result later by webhook.
2. Fully synchronous: include ledger posting and notifications inside the same request.

Rationale: Checkout needs an immediate answer, and card networks respond within seconds. Only authentication, tokenization, the idempotency check, fraud scoring, network authorization and the state commit are on the critical path, with an estimated budget of about 220 milliseconds. Treating a timeout as pending avoids declaring a payment failed when the network may have approved it.

Trade-offs: The availability of the payment path depends on every synchronous dependency, so each needs a timeout, a circuit breaker and a fallback. The latency budget is tight, and each added step spends part of it. A pending payment needs a resolution job that queries the network.

Consequences: Deadlines propagate end to end so no work continues after the client times out. Idempotency keys are mandatory so retries never charge twice. Fraud scoring needs a degraded mode (local rules) when its circuit opens. Any step proposed for the critical path must justify its share of the budget.

7.4 Architecture Decision Record 4: Distributed relational storage for payments and the ledger; key-value and columnar stores elsewhere

Decision: Payments, refunds and the ledger use sharded, consensus-replicated relational storage with full transactional guarantees, while idempotency, token and fraud-feature lookups use key-value storage and analytics use columnar storage.

Context:

- Non-Functional Requirement 07 requires a Recovery Point Objective of zero for acknowledged financial transactions.
- Functional Requirement 02 requires payment status transitions that are never lost or applied twice.
- Functional Requirements 07, 13 and 14 require exact balances and reconciliation.
- Workload Parameter 04 gives 35 percent reads and 65 percent writes, so the transactional layer is write-heavy.

Alternatives Considered:

1. Non-relational storage (document or wide-column) for everything: eventual consistency and easy horizontal scale.
2. A single-node relational database: strong guarantees but limited to one machine.

Rationale: A double-entry journal needs atomic multi-row writes in which debits equal credits, and a uniqueness constraint on the source event so a replayed event cannot post twice. Sharding by seller identifier keeps each seller's transactions inside one shard, so the guarantees hold without cross-shard transactions. Non-relational storage remains the right fit where eventual consistency is acceptable, such as velocity counters and idempotency records with a time to live.

Trade-offs: Quorum commits add latency (the budget allows about 30 milliseconds). A relational shard sustains fewer writes per node than a key-value store, so more shard groups are needed; the sizing is a design assumption that must be validated by load testing. Operating several storage models adds cost.

Consequences: Transactions must stay within one seller's shard, which constrains the data model. Very large sellers need sub-partitioning and hot-account sub-accounts. Analytics must not query the transactional stores, which leads to Record 5. Schema changes on large financial tables must be planned carefully.

7.5 Architecture Decision Record 5: Command Query Responsibility Segregation with event-fed read models

Decision: The system separates the transactional write model from read models fed by events, and serves reads of very recent objects from the owning store.

Context:

- Functional Requirement 03 and Use Case 02 require the current payment status for authorized Buyers and Sellers.
- Functional Requirement 07 and Use Case 06 require dashboards with balance and settlement information.
- Functional Requirements 13 and 14 and Use Case 10 require period-wide reconciliation views.
- Non-Functional Requirement 05 limits synchronous requests to 500 milliseconds at the 95th percentile, and Non-Functional Requirement 03 sets the peak load.

- Workload Parameter 04 shows reads are the minority by count (35 percent), but dashboards, search and reports are heavier per request than payment writes.

Alternatives Considered:

1. One shared model: dashboards and reports query the transactional stores directly.
2. Read replicas of the transactional stores: same schema, different nodes.
3. A cache in front of the transactional stores.

Rationale: Expensive queries must never compete with the payment path. Projections can be shaped for each use case (seller dashboard, transaction search, auditor view) and scaled independently. Because events already exist (Record 2), read models come almost for free.

Trade-offs: Read models lag the write model (target under 5 seconds), so users may see slightly stale data. Data is stored twice, and projections must be maintained and rebuilt when schemas change. Two read paths exist: read models for general views and the owning store for just-written objects.

Consequences: A request for an object changed in the last few seconds must be served by the owning store to give read-your-writes behavior. New views can be added by replaying events. The product must accept and document eventual consistency in dashboards. Decisions that move money must never rely on read models.

7.6 Architecture Decision Record 6: Orchestrated sagas for refunds and disputes instead of distributed transactions

Decision: Refunds and disputes are coordinated by an orchestrated saga with persisted state and compensating actions, rather than by a distributed transaction.

Context:

- Use Case 04 and Functional Requirement 05 require refunds through the payment network; alternative flow 4a handles a network rejection.
- Use Case 07 and Functional Requirements 08 and 09 require a dispute lifecycle with deadlines and evidence that lasts days.
- Non-Functional Requirement 07 requires no loss of acknowledged financial state.
- The banking and payment networks are external systems that cannot join our transaction.

Alternatives Considered:

1. Two-phase commit across the payment store, the ledger and the network.
2. Choreographed saga: services react to each other's events with no central coordinator.

Rationale: An external network cannot take part in a two-phase commit, and holding locks across long-running flows would harm availability. Orchestration keeps an explicit, queryable state for each refund or dispute, which Administrators (Use Case 07) and Auditors can inspect. Compensation logic is centralized, for example a reversing journal that releases the reserved amount if the network rejects the refund.

Trade-offs: Intermediate states are visible (for example, an amount reserved but not yet refunded). Every step needs a designed compensating action. The orchestrator is a critical component whose failure stops in-flight flows until it recovers.

Consequences: Every saga step must be idempotent because steps may be retried. The saga state machine is part of the data model of the Refund and Dispute Store and must be backed up with it. Timeouts and deadlines for disputes must be modeled explicitly. Adding a step means adding both its action and its compensation.

8 Requirements Traceability

This table maps each functional requirement from Delivery 1 to the specific service(s) responsible for satisfying it in the proposed architecture.

Table 27: Requirements Traceability

ID	Functional Requirement	Responsible Service(s)
FR-01	Validate payment information before processing a payment request.	Payment Orchestrator
FR-02	Process payment requests and maintain their corresponding transaction status.	Payment Orchestrator
FR-03	Provide the current status of a payment transaction to an authorized Buyer or Seller.	Payment Orchestrator, Read Models and Query Service
FR-04	Analyze payment transactions for potentially fraudulent activity before completing the transaction.	Fraud Scoring Service
FR-05	Allow an authorized Seller to submit and process refund requests for eligible completed payments.	Refund and Dispute Service
FR-06	Generate and deliver notifications for relevant payment events to the Seller or Seller's System.	Notification and Webhook Dispatcher
FR-07	Display to an authorized Seller the available balance and the settlement information associated with their account.	Ledger Service, Read Models and Query Service
FR-08	Allow an authorized Seller to view the list of payment disputes and the details of each disputed transaction.	Refund and Dispute Service, Read Models and Query Service

ID	Functional Requirement	Responsible Service(s)
FR-09	Allow an authorized Seller to submit a response and supporting information for a dispute.	Refund and Dispute Service
FR-10	Provide an authenticated and authorized Administrator with an interface displaying payment and operational information.	Monitoring and Audit Service
FR-11	Allow a registered Seller to submit the information required for verification.	Authentication Service
FR-12	Allow an authorized Administrator to review a verification request and approve it, reject it, request additional information, or keep it pending.	Authentication Service
FR-13	Allow an authorized Auditor to select the transactions or period to be reconciled and display the results.	Settlement and Reconciliation Service
FR-14	Identify and flag transactions with missing settlement information or discrepancies.	Settlement and Reconciliation Service